Published: 12 July 2024
Accepted: 23 January 2024
Received: 29 September 2023

RESEARCH-ARTICLE

CodePlan: Repository-Level Coding using LLMs and Planning

RAMAKRISHNA BAIRI, Microsoft Research, Redmond, WA, United States

ATHARV SONWANE, Microsoft Research, Redmond, WA, United States

ADITYA KANADE, Microsoft Research, Redmond, WA, United States

VAGEESH D. C., Microsoft Research, Redmond, WA, United States

ARUN IYER, Microsoft Research, Redmond, WA, United States

SURESH PARTHASARATHY, Microsoft Research, Redmond, WA, United States

View all

Proceedings of the ACM on Software Engineering, Volume 1, Issue FSE (July 2024)
https://doi.org/10.1145/3643757
EISSN: 2994-970X

CodePlan: Repository-Level Coding using LLMs and Planning

RAMAKRISHNA BAIRI, Microsoft Research, India
ATHARV SONWANE, Microsoft Research, India
ADITYA KANADE, Microsoft Research, India
VAGEESH D. C., Microsoft Research, India
ARUN IYER, Microsoft Research, India
SURESH PARTHASARATHY, Microsoft Research, India
SRIRAM RAJAMANI, Microsoft Research, India
B. ASHOK, Microsoft Research, India
SHASHANK SHET, Microsoft Research, India

Software engineering activities such as package migration, fixing error reports from static analysis or testing, and adding type annotations or other specifications to a codebase, involve pervasively editing the entire repository of code. We formulate these activities as *repository-level coding* tasks.

Recent tools like GitHub Copilot, which are powered by Large Language Models (LLMs), have succeeded in offering high-quality solutions to localized coding problems. Repository-level coding tasks are more involved and cannot be solved directly using LLMs, since code within a repository is inter-dependent and the entire repository may be too large to fit into the prompt. We frame repository-level coding as a planning problem and present a task-agnostic, neuro-symbolic framework called CodePlan to solve it. CodePlan synthesizes a multi-step chain-of-edits (plan), where each step results in a call to an LLM on a code location with context derived from the entire repository, previous code changes and task-specific instructions. CodePlan is based on a novel combination of an incremental dependency analysis, a change may-impact analysis and an adaptive planning algorithm (symbolic components) with the neural LLMs.

We evaluate the effectiveness of CodePlan on two repository-level tasks: package migration (C#) and temporal code edits (Python). Each task is evaluated on multiple code repositories, each of which requires inter-dependent changes to many files (between 2–97 files). Coding tasks of this level of complexity have not been automated using LLMs before. Our results show that CodePlan has better match with the ground truth compared to baselines. CodePlan is able to get 5/7 repositories to pass the validity checks (i.e., to build without errors and make correct code edits) whereas the baselines (without planning but with the same type of contextual information as CodePlan) cannot get any of the repositories to pass them. We provide our (non-proprietary) data, evaluation scripts and supplementary material at https://github.com/microsoft/codeplan.

CCS Concepts: • **Computing methodologies** → **Planning under uncertainty**; • **Software and its engineering** → **Software maintenance tools**; **Software evolution**; **Automatic programming**.

Additional Key Words and Phrases: Automated coding, repositories, LLMs, static analysis, plan, chain of edits, neuro-symbolic AI

Authors' addresses: Ramakrishna Bairi, Microsoft Research, India, rbairi@microsoft.com; Atharv Sonwane, Microsoft Research, India, t-asonwane@microsoft.com; Aditya Kanade, Microsoft Research, India, kanadeaditya@microsoft.com; Vageesh D. C., Microsoft Research, India, vachand@microsoft.com; Arun Iyer, Microsoft Research, India, ariy@microsoft.com; Suresh Parthasarathy, Microsoft Research, India, supartha@microsoft.com; Sriram Rajamani, Microsoft Research, India, sriram@microsoft.com; B. Ashok, Microsoft Research, India, bash@microsoft.com; Shashank Shet, Microsoft Research, India, t-sshet@microsoft.com.

ACM 2994-970X/2024/7-ART31
https://doi.org/10.1145/3643757

ACM Reference Format:
Ramakrishna Bairi, Atharv Sonwane, Aditya Kanade, Vageesh D. C., Arun Iyer, Suresh Parthasarathy, Sriram Rajamani, B. Ashok, and Shashank Shet. 2024. CodePlan: Repository-Level Coding using LLMs and Planning. *Proc. ACM Softw. Eng.* 1, FSE, Article 31 (July 2024), 24 pages. https://doi.org/10.1145/3643757

Fig. 1. Task instruction to migrate a code repository due to an API change in the Complex Numbers library.

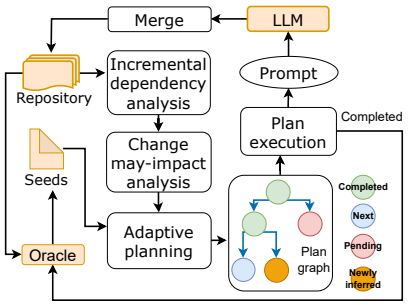


Fig. 2. Overview of CodePlan.

1 INTRODUCTION

The remarkable generative abilities of Large Language Models (LLMs) [30, 34, 36, 41, 64, 81] have opened new ways to automate coding tasks. Tools built on LLMs, such as Amazon Code Whisperer [3], GitHub Copilot [7] and Replit [13], are now widely used to complete code given a natural language intent and context of surrounding code, and also to perform code edits based on natural language instructions [6]. Such edits are typically done for small regions of code such as completing or editing the current line, or the body of the entire method.

While these tools help with the "inner loop" of software engineering where the developer is editing a small region of code, there are several tasks in the "outer loop" of software engineering that involve the entire code repository. For example, if our repository uses a library L, and its API changes from version v_n to version v_{n+1}, we need to migrate our repository to correctly invoke the revised version. Such a migration task involves making edits not only to all the regions of repository that make calls to the APIs in library L, but also to regions of the repository (across file boundaries) having transitive syntactic and semantic dependencies on the updated code.

This is illustrated in Fig. 1, which shows a change in the API for a Complex Numbers library. Our task is to migrate the code repository in accordance with this change. The left side of Fig. 3 shows relevant parts: The file `Create.cs` has the method `func`, which invokes the `create_complex` method from the library, and `Process.cs` has the method `process` which invokes `func`.

We can pass the task description from Fig. 1 and the body of `func` to an LLM to generate the revised code for `func` as shown in the right side of Fig. 3. As seen, the LLM has correctly edited the invocation to the `create_complex` API so that it returns an object of type `Complex` instead of a tuple of two floating point values. Note that this edit has resulted in a change to the signature of the method `func` – it now returns an object of type `Complex`. This necessitates changes to callers of method `func` such as the `process` method in file `Process.cs`, shown in the left-bottom of Fig. 3. Without a suitable change to the body of the `process` method, our code does not build! The bottom-right of Fig. 3 shows a suitable change to the `process` method that gets the repository to a valid state where it builds without errors.

Problem Formulation. The migration task above is representative of a family of tasks that involve editing an entire code repository for various purposes such as fixing error reports from static analysis or testing, fixing a buggy coding pattern, refactoring, or adding type annotations or other

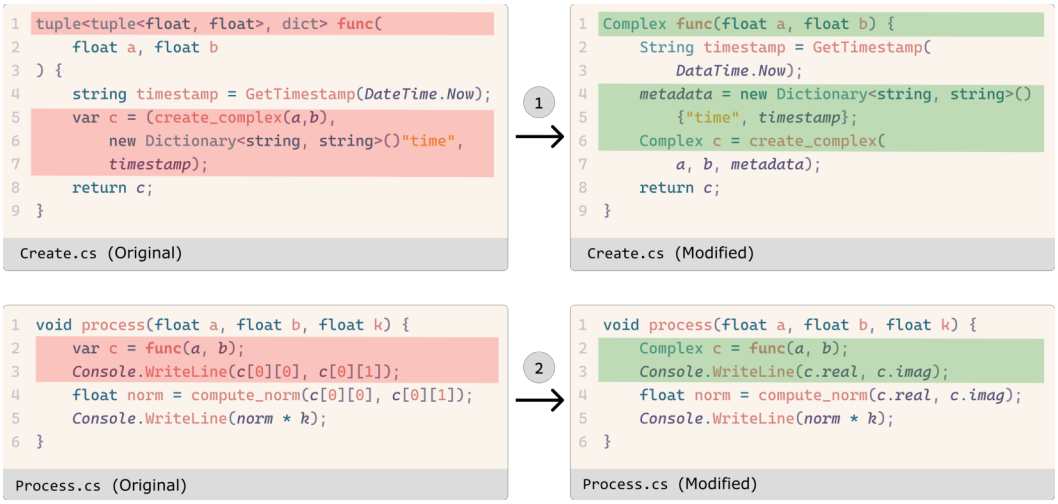


Fig. 3. Changes to other files resulting from the seed change specified in Fig 1

specifications. Each of these tasks involves a set of *seed specifications* such as the one shown in Fig. 1, which are starting points for the code editing task. These seed specifications typically trigger other editing requirements which need to be propagated across dependencies in the code repository. Typically, such propagation of edits across dependencies is done manually.

Our goal is to construct a repository-level coding system, which automatically generates *derived specifications* for edits such as one required for the `process` method in Fig. 3, in order to get the repository to a *valid* state. Here, validity is defined with respect to an oracle, which can be instantiated to various ways of enforcing repository-level correctness conditions such as building without errors, passing static analysis, passing a type system or a set of tests, or passing a verification tool. We define an LLM-driven repository-level coding task as follows:

LLM-driven Repository-level Coding Task

Given a start state of a repository R_{start}, a set of seed edit specifications Δ_{seeds}, an oracle Θ s.t. $\Theta(R_{start})$ = True, and an LLM L, the goal of an **LLM-driven repository-level coding task** is to reach a state $R_{target} = ExecuteEdits(L, R_{start}, P)$ where P is a chain of edit specifications from $\Delta_{seeds} \cup \Delta_{derived}$ and $\Delta_{derived}$ is a set of derived edit specifications so that $\Theta(R_{target})$ = True.

Proposed Solution. In this paper, we propose a method to compute derived specifications by framing (LLM-driven) repository-level coding as a *planning problem*. Automated planning [43, 75] aims to solve multi-step problems, where each step executes one action among many alternatives towards reaching a target state. It is used in a wide range of areas such as motion planning [54], autonomous driving [44], robotics [51] and theorem proving [32].

We present a task-agnostic framework, called CodePlan, which synthesizes a multi-step plan to solve the repository-level coding task. As shown in Fig. 2, the input to CodePlan is (1) a repository, (2) a task with seed specifications expressed through a natural language instruction or a set of manual code edits, (3) a correctness oracle and (4) an LLM capable of editing code given instructions. CodePlan constructs a *plan graph* where each node in the graph identifies a code edit obligation that the LLM needs to discharge and an edge indicates that the target node needs to be discharged consequent to the source node. CodePlan monitors the code edits and adaptively extends the plan graph. The edits Δ_{seeds} follow from the task description, whereas the edits $\Delta_{derived}$ are identified and

contextualized based on a novel combination of an incremental dependency analysis, a change may-impact analysis and an adaptive planning algorithm. The merge block merges the code generated by the LLM into the repository. Once all the steps in a plan are completed, the repository is analyzed by the oracle. The task is completed if the oracle validates the repository. If it finds errors, the error reports are used as seed specifications for the next iteration of plan generation and execution. CodePlan is a neuro-symbolic framework [47] which combines the neural LLMs with symbolic components based on static analysis and planning.

Consider again, the example API migration task specified in Fig. 1 on code in Fig. 3. CodePlan performs the edit of the method `func` using the instruction in Fig. 1 as a seed specification. By analyzing the code change between Fig. 3(a)–(b), it classifies the change as an *escaping change* as it affects signature of method `func`. The change may-impact analysis identifies that the caller(s) of `func` may be affected and hence, the adaptive planning algorithm uses caller-callee dependencies to infer a derived specification to edit the method `process`, which invokes `func`. Both the seed and derived changes are executed by creating suitable prompts for an LLM and the resulting code repository passes the oracle, i.e., builds without errors. Note that this is a simple example with only one-hop change propagation. In practice, the derived changes can themselves necessitate other changes transitively and CodePlan handles such cases.

Contributions. To the best of our knowledge, the problem of monitoring the effects of code edits made by an LLM to a repository and systematically planning *a chain of inter-dependent edits* has not been identified and solved before. At repository-level scope, two types of contexts have been found to be useful for prompting LLMs: (1) *spatial context* to provide cross-file information to the model using static analysis [15, 40, 58, 67, 69, 78, 79, 86] or retrieval [90, 94], and (2) *temporal context* to condition the predictions on the history of edits to the repository [29, 45, 72, 85]. Since CodePlan monitors the code changes and maintains a repository-wide dependency graph, we provide both these forms of contexts in a unified framework. The existing techniques assume that the next edit location is provided by the developer and do not account for the effect of an edit on the dependent code. In contrast, by inferring the impact of each change, CodePlan propagates the changes to dependent code, paving a way to automate a repository-level coding task through a chain-of-edits.

In summary, we make the following contributions in this paper:

(1) We are the first to formalize the problem of automating repository-level coding tasks using LLMs, which requires analyzing the effects of code changes and propagating them across the repository. There are currently no systematic and scalable solutions to this problem.
(2) We frame repository-level coding as a planning problem and design a task-agnostic, neuro-symbolic framework called CodePlan, based on a novel combination of an incremental dependency analysis, a change may-impact analysis and an adaptive planning algorithm. CodePlan synthesizes a multi-step chain-of-edits (plan) to be actuated by an LLM.
(3) We experiment with two repository-level coding tasks: package migration for C# and temporal code edits for Python repositories, using `gpt-4-32k` model [8]. To demonstrate the ability of CodePlan to work with different models, we evaluate two additional open-source models: CodeLlama [74] and Coeditor (a fine-tuned code-editing model) [85]. We compare against baselines that use build system or type checker to guide repository-wide edits. For fair comparison, we use the same contextualization method as CodePlan in the baselines.
(4) Our results show that CodePlan has better match with the ground truth compared to baselines. CodePlan is able to get 5/7 repositories to pass the validity checks (i.e., to build without errors and make correct code edits), whereas the baselines cannot get any of the repositories to pass them. Except for the 2 proprietary repositories, we are making our data, evaluation scripts and supplementary material available at https://github.com/microsoft/codeplan.

2 DESIGN

In this section, we first give an overview of the CodePlan algorithm for automating repository-level coding tasks (Section 2.1). We then present the static analysis (Section 2.2) and the adaptive planning and plan execution (Section 2.3) components of CodePlan.

2.1 The CodePlan Algorithm

```
1  /* Inputs: R is the source code of a repository, Delta_seeds is a set of seed edit
       specifications, Theta is an oracle and L is an LLM. */

3  CodePlan(R, Delta_seeds, Theta, L):
4    let mutable G: PlanGraph = null in
5    let mutable D: DependencyGraph = ConstructDependencyGraph(R) in
6      while Delta_seeds is not empty
7        IntializePlanGraph(G, Delta_seeds)
8        AdaptivePlanAndExecute(R, D, G)
9        Delta_seeds := ExtractSeeds(Theta(R))

11 InitializePlanGraph(G, Delta_seeds):
12   for each ⟨B, I⟩ in Delta_seeds
13     AddRoot(G, ⟨B, I, Pending⟩)

15 AdaptivePlanAndExecute(R, D, G):
16   while G has Nodes with Pending status
17     let ⟨B, I, Pending⟩ = GetNextPending(G) in
18     // First step: extract fragment of code
19     let Fragmemt = ExtractCodeFragment(B, R) in
20     // Second step: gather context of the edit
21     let Context = GatherContext(B, R, D) in
22     // Third step: use the LLM to get edited code fragment
23     let Prompt = MakePrompt(Fragment, I, Context) in
24     let NewFragment = InvokeLLM(L, Prompt) in
25     // Fourth step: merge the updated code fragment into R
26     let R := Merge(NewFragment, B, R) in
27     let Labels = ClassifyChanges(Fragment, NewFragment) in
28     let D' = UpdateDependencyGraph(D, Labels, Fragment, NewFragment, B) in
29     // Fifth step: adaptively plan and propogate the effect of the edit on dependant code
30     let BlockRelationPairs= GetAffectedBlocks(Labels, B, D, D') in
31       MarkCompleted(B, G)
32       for each ⟨B', rel⟩ in BlockRelationPairs
33         let N = GetNode(B) in
34         let M = SelectOrAddNode(B', Nil, Pending) in
35           AddEdge(G, M, N, rel)
36     D := D'

38 GatherContext(B, R, D):
39   let SC = GetSpatialContext(B, R) in
40   let TC = GetTemporalContext(G, B) in
41     (SC, TC)
```

Algorithm 1: The CodePlan algorithm to automate repository-level coding tasks. The data structures and functions in Cyan and Orchid are explained in Sections 2.2 and 2.3 respectively.

The CodePlan algorithm (Algorithm 1) takes four inputs: (1) the source code of a repository R, (2) a set of seed edit specifications for the task in hand, Δ_{seeds}, (3) an oracle, Θ, that can determine correctness of the repository, and (4) an LLM, L.

The core data structure maintained by the algorithm is a *plan graph* G, a directed acyclic graph with multiple root nodes (line 4). Each node in the plan graph is a tuple $\langle B, I, Status \rangle$, where B is a block of code (that is, a sequence of code locations) in the repository R, I is an edit instruction (along the lines of the example shown in Fig. 1), and $Status$ is either *pending* or *completed*.

The CodePlan algorithm also maintains a *dependency graph* D (line 5). Fig. 4 illustrates the dependency graph structure. We will discuss it in details in Section 2.2.1. For now, it suffices to

know that the dependency graph D represents the syntactic and semantic dependency relations between code blocks in the repository R.

The loop at lines 6–9 is executed until Δ_{seeds} is empty. Line 7 calls the `InitializePlanGraph` function (lines 11–13) that adds all the changes in Δ_{seeds} as root nodes of the plan graph. Each edit specification consists of a code block B and an edit instruction I. The status is set to pending for the root nodes (line 13). The function `AdaptivePlanAndExecute` is called at line 8 which executes the plan, updates the dependency graph with each code change and extends the plan as necessary. Once the plan is completely executed, the oracle Θ is run on the repository. It returns error locations and diagnostic messages from which `ExtractSeeds` extracts Δ_{seeds} for the next iteration. If the repository passes the oracle's checks then it returns an empty set and CodePlan terminates.

We now discuss `AdaptivePlanAndExecute`, which is the main work horse. It iteratively picks each pending node and processes it. Processing a pending node for a block B with edit instruction I involves the following five steps:

(1) **The *first step* (line 19) is to extract the fragment of code to edit.** Simply extracting code of the block B loses information about relationship of B with the surrounding code. Keeping the entire file on the other hand takes up prompt space and is often unnecessary. We found the surrounding context is most helpful when a block belongs to a class. For such blocks, we sketch the enclosing class. That is, in addition to the code of block B, we also keep declarations of the enclosing class and its members. As we discuss later, this sketched representation also helps us merge the LLM's output into a source code file more easily.

(2) **The *second step* (line 21) is to gather the context of the edit.** The context of the edit (line 38–41) consists of (a) *spatial context*, which contains related code such as methods called from the block B, and (b) *temporal context*, which contains the previous edits that *caused* the need to edit the block B. The temporal context is formed by edits along the paths from the root nodes of the plan graph to B.

(3) **The *third step* (lines 23–24) constructs a prompt** using the fragment extracted in the first step, the instruction I from the edit specification and the context extracted in the second step, and **invokes the LLM using the prompt** to get the edited code fragment.

(4) **The *fourth step* (lines 26–28) merges the edited code back into the repository.** Since the code is updated, many dependency relationships such as caller-callee, class hierarchy, etc. may need to change, and hence, **this step also updates the dependency graph D.**

(5) **The *fifth and final step* (lines 30–35) does adaptive planning to propagate the effects of the current edit on dependant code blocks.** This involves classifying the change in the edited block, and depending on the type of change, picking the right dependencies in the dependency graph to traverse and locate affected blocks. For instance, if the edit of a method m in the current block B involves update to the signature of the method, then all callers of m get affected (the scenario in Fig. 3). For each affected block B' and the dependency relation `rel` connecting B to B' in the dependency graph, we get a pair $\langle B', \texttt{rel} \rangle$. If a node exists for B' in the plan graph and it is pending, then we add an edge from B to B' labeled with `rel` to the plan graph. Otherwise, the edge is added to a newly created node for B' (line 34). The block B is marked as completed (line 31).

CodePlan selects pending nodes in breadth-first order, but other scheduling choices may be used. In general, CodePlan is robust to the order in which pending nodes are evaluated. Suppose code A that has been used as context in an edit to code B undergoes changes in future. The iterative nature of CodePlan ensures that a change obligation is created for code B subsequent to that.

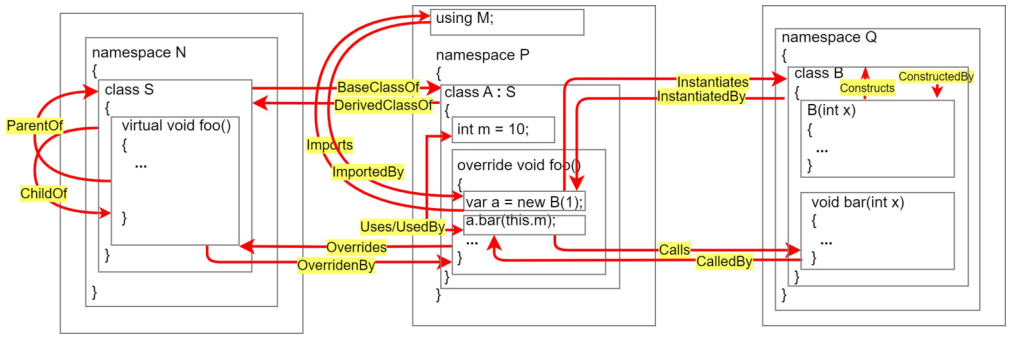


Fig. 4. Illustration of the dependency graph annotated with relations as the edge labels.

2.2 Static Analysis Components

We now turn our attention to the static analysis components used in CodePlan. We will cover all the data structures and functions in Cyan background from Algorithm 1.

2.2.1 Incremental Dependency Analysis. An LLM can be provided a code fragment and an instruction to edit it in a prompt. While the LLM may perform the desired edit accurately, analyzing the impact of the edit on the rest of the repository is outside the scope of the LLM call. We believe static analysis is well-suited to do this and propose an incremental dependency analysis for the same.

`DependencyGraph`. Dependency analysis [18] is used for tracking syntactic and semantic relations between code elements. In our case, we are interested in relations between import statements, methods, classes, field declarations and statements (excluding those that operate only on variables defined locally within the enclosing method). Formally, a *dependency graph* $\mathrm{D} = (N, E)$ where N is a set of nodes representing the code blocks mentioned above and E is a set of labeled edges where the edge label gives the relation between the source and target nodes of the edge. Fig. 4 illustrates all the relations we track. The relations include (1) *syntactic relations* (ParentOf and ChildOf, Constructs and ConstructedBy) between a block c and the block p that encloses c syntactically; a special case being a constructor and its enclosing class related by Constructs and ConstructedBy, (2) *import relations* (Imports and ImportedBy) between an import statement and statements that use the imported modules, (3) *inheritance relations* (BaseClassOf and DerivedClassOf) between a class and its superclass, (4) *method override relations* (Overrides and OverridenBy) between an overriding method and the overridden method, (5) *method invocation relations* (Calls and CalledBy) between a statement and the method it calls, (6) *object instantiation relations* (Instantiates and InstantiatedBy) between a statement and the constructor of the object it creates, and (7) *field use relations* (Uses and UsedBy) between a statement and the declaration of a field it uses.

`ConstructDependencyGraph`. The dependency relations are derived across the source code spread over the repository through static analysis. We represent the source code of a repository as a forest of abstract syntax trees (ASTs) and add the dependency edges between AST sub-trees. A file-local analysis derives the syntactic and import relations. All other relations require an inter-class, inter-procedural analysis that can span file boundaries. In particular, we use the class hierarchy analysis [38] for deriving the semantic relations.

`ClassifyChanges`. As discussed in Section 2.1, in the fourth step, CodePlan merges the code generated by the LLM into the repository. By pattern-matching the code before and after, we classify the code changes. Table 1 (the first column) gives the type of atomic change. Broadly, the changes are organized as modification, addition and deletion changes, and further by which construct is changed. We distinguish between method body and method signature changes. Similarly, we distinguish between changes to a class declaration, to its constructor or to its fields. The changes to

Table 1. Rules for updating the dependency graph and for change may-impact analysis for atomic changes. We refer to the dependency graphs before and after the updates by D and D′ respectively.

Atomic Change	Dependency Graph Update	Change May-Impact Analysis
Modification Changes		
Body of method M	Recompute the edges incident on the statements in the method body.	If an escaping object is modified then Rel(D, M, CalledBy) else Nil.
Signature of method M	Recompute the edges incident on the method.	Rel(D, M, CalledBy), Rel(D, M, Overrides), Rel(D, M, OverriddenBy), Rel(D′, M, Overrides), Rel(D′, M, OverriddenBy)
Field F in class C	Recompute the edges incident on the field.	Rel(D, F, UsedBy), Rel(D, C, ConstructedBy), Rel(D, C, BaseClassOf), Rel(D, C, DerivedClassOf)
Declaration of class C	Recompute the edges incident on the class.	Rel(D, C, InstantiatedBy), Rel(D, C, BaseClassOf), Rel(D, C, DerivedClassOf), Rel(D′, C, BaseClassOf), Rel(D′, C, DerivedClassOf)
Signature of constructor of class C	No change.	Rel(D, C, InstantiatedBy), Rel(D, C, BaseClassOf), Rel(D, C, DerivedClassOf)
Import/Using statement I	Recompute the edges incident on the import statement.	Rel(D, I, ImportedBy)
Addition Changes		
Method M in class C	Add new node and edges by analyzing the method. If C.M overrides a base class method B.M then redirect the Calls/-CalledBy edges from B.M to C.M if the receiver object is of type C.	Rel(D, C, BaseClassOf), Rel(D, C, DerivedClassOf), Rel(D′, M, CalledBy)
Field F in class C	Add new node and edges by analyzing the field declaration.	Rel(D, C, ConstructedBy), Rel(D, C, BaseClassOf), Rel(D, C, DerivedClassOf)
Declaration of class C	Add new node and edges by analyzing the class declaration.	Nil
Constructor of class C	Add new node and edges by analyzing the constructor.	Rel(D, C, InstantiatedBy), Rel(D, C, BaseClassOf), Rel(D, C, DerivedClassOf)
Import/Using statement I	Add new node and edges by analyzing the import statement.	Nil
Deletion Changes		
Method M in class C	Remove the node for M and edges incident on M. If C.M overrides a base class method B.M then redirect the Calls/CalledBy edges from C.M to B.M if the receiver object is of type C.	Rel(D, M, CalledBy), Rel(D, M, Overrides), Rel(D, M, OverriddenBy)
Field F in class C	Remove the node of the field and edges incident on it.	Rel(D, F, UsedBy), Rel(D, C, ConstructedBy), Rel(D, C, BaseClassOf), Rel(D, C, DerivedClassOf)
Declaration of class C	Remove the node of the class and edges incident on it.	Rel(D, C, InstantiatedBy), Rel(D, C, BaseClassOf), Rel(D, C, DerivedClassOf)
Constructor of class C	Remove edges to the class due to object instatiations using the constructor.	Rel(D, C, InstantiatedBy), Rel(D, C, BaseClassOf), Rel(D, C, DerivedClassOf)
Import/Using statement I	Remove the node of the import statement and edges incident on it.	Rel(D, I, ImportedBy)

import statements or the statements that use imports are also identified. These are *atomic changes*. An LLM can make multiple simultaneous edits in the given code fragment, resulting in multiple atomic changes, all of which are identified by the `ClassifyChanges` function.

`UpdateDependencyGraph`. As code generated by the LLM is merged, the dependency relations associated with the code at the change site are re-analyzed. Table 1 (the second column) gives the rules to update the dependency graph D to D′ based on the labels inferred by `ClassifyChanges`. For modification changes, we recompute the relations of the changed code except for constructors.

A constructor is related to its enclosing class by a syntactic relation which does not have to be recomputed. For addition changes, new nodes and edges are created for the added code. Edges corresponding to syntactic relations are created in a straightforward manner. If a change simultaneously adds an element (an import, a method, a field or a class) and its uses, we create a node for the added element before analyzing the statements that use it. Addition of a method needs special handling as shown in the table: if an overriding method C.M is added then the Calls/CalledBy edges incident on the matching overridden method B.M are redirected to C.M if the call is issued on a receiver object of type C. The deletion of an overriding method requires an analogous treatment as stated in Table 1. All other deletions require removing nodes and edges as stated in the table.

2.2.2 Change May-Impact Analysis. In the fifth step, CodePlan identifies the code blocks that may have been impacted by the code change by the LLM. Let Rel(D, B, rel) be the set of blocks that are connected to a block B via relation rel in the dependency graph D. Let D and D′ be the dependency graph before and after the updates in Table 1.

`GetAffectedBlocks`. The last column in Table 1 tells us how to identify blocks affected by a code change. When the body of a method M is edited, we perform escape analysis [28, 35] to identify if any object accessible in the callers of M (an escaping object) has been affected by the change. If yes, the callers of M (identified through Rel(D, M, CalledBy)) are identified as affected blocks. Otherwise, the change is localized to the method and no blocks are affected. If the signature of a method is edited, the callers and methods related to it through method-override relations in the inheritance hierarchy are affected. The signature change can affect the Overrides and OverridenBy relations themselves, e.g., addition or deletion of the `@Override` access modifier. Therefore, the blocks related by these relations in the updated dependency graph D′ are also considered as affected as shown in Table 1. When a field F of a class C is modified, the statements that use F, the constructors of C and sub/super-classes of C are affected. When a class is modified, the methods that instantiate it and its sub/super-classes as per D and D′ are affected. A modification to a constructor has a similar rule except that such a change does not change inheritance relations and hence, only D is required. When an import statement I is modified, the statements that use the imported module are affected.

The addition and deletion changes are less complex than the modification changes, and their rules are designed along the same lines as discussed above. In the interest of space, we do not explain each of them step-by-step. We assume that a new class or import is not used *before* it is added, however it may be used in a future edit. Since there is no use that exists (at the time of addition), no code block is affected in change may-impact analysis for these additions. In our experiments, we have found the rules in Table 1 to be adequate. However, CodePlan can be easily configured to accommodate extensions of the rules in Table 1 if necessary.

2.3 Adaptive Planning and Plan Execution

We now discuss the data structures and functions from Algorithm 1 in the Orchid background.

2.3.1 Adaptive Planning. Having identified the affected blocks (using `GetAffectedBlocks`), CodePlan creates change obligations that need to be discharged using an LLM to make the dependent code consistent with the change. As discussed in Section 2.1, this is an iterative process.

`PlanGraph`. A *plan graph* P = (O, C) is a directed acyclic graph with a set of *obligations* O, each of which is a triple $\langle B, I, status \rangle$ where B is a block, I is an instruction and status is either pending or completed. An edge in C records the *cause*, the dependency relation between the blocks in the source and target obligations. In other words, the edge label identifies which Rel clause in a change may-impact rule in Table 1 results in creation of the target obligation.

`ExtractCodeFragment`. As discussed in the first step in Section 2.1, simply extracting code for a block B is sub-optimal as it loses context. The `ExtractCodeFragment` function takes the whole class

the code block belongs to, keeps the complete code for B and retains only declarations of the class and other class members. Empirically, we found this to be useful because the names and types of the class and other members provide additional context to the LLM. Often times the LLM needs to make multiple simultaneous changes. For example, in some of our case studies, the LLM has to add a field declaration, take an argument to a constructor and use it within the constructor to initialize the field. Providing the sketch of the surrounding code as a code fragment to the LLM allows the LLM to make these changes at the right places. The code fragment extraction logic is implemented by traversing the AST and "folding" away the subtrees (e.g., method bodies) that are sketched. This reduces the code size without sacrificing naturalness of code [46]. As stated in Section 1, this sketched representation also allows us to place the LLM generated code back into the AST without ambiguity, even when there are multiple simultaneous changes.

`GetSpatialContext`. Spatial context in CodePlan refers to the arrangement and relationships of code blocks within a codebase, helping understand how classes, functions, variables, and modules are structured and interact. It's crucial for making accurate code changes. CodePlan utilizes the dependency graph to extract spatial context. This enables CodePlan to make context-aware code modifications that are consistent with the code's spatial organization, enhancing the accuracy and reliability of its code editing capabilities. In particular, when generating an edit to a method, CodePlan fetches all the methods called in the body of the method to be edited, class members accessed, along with methods that override or are overridden by the method to be edited. For constructors, we fetch the constructor of super-class if present.

`GetTemporalContext`. The plan graph records all change obligations and their inter-dependences. Extracting temporal context is accomplished by linearizing all paths from the root nodes of the plan graph to the target node. Each change is a pair of the code fragments before and after the change. The temporal context also states the "causes" (recorded as edge labels) that connect the target node with its predecessor nodes. For example, if a node A is connected to B with a CalledBy edge, then the temporal context for B is the before/after fragments for A and a statement that says that "B calls A", which helps the LLM understand the cause-effect relation between the latest temporal change (change to A) and the current obligation (to make a change to B).

2.3.2 Plan Execution. CodePlan iteratively selects a pending node in the plan graph and invokes an LLM to discharge the change obligation.

p_1 Task Instructions: *Your task is to . . .*
p_2 Earlier Code Changes: *These are edits that have been made in the code-base previously -*
p_3 Causes for Change: *The change is required due to -*
p_4 Related Code: *The following code maybe related -*
p_5 Code to be Changed Next: *The existing code is given below -*

Edit the "Code to be Changed Next" and produce "Changed Code" below. Edit the "Code to be Changed Next" according to the "Task Instructions" to make it consistent with the "Earlier Code Changes", "Causes for Change" and "Related Code". If no changes are needed, output "No changes."

`MakePrompt`. Having extracted the code fragment to be edited along with the relevant spatial and temporal context, we construct a prompt to pass to the LLM with the structure given below. We open with the task specific instructions p_1 followed by listing the edits made in the repository so far p_2 that are relevant to the fragment being edited (temporal context). The next section p_3 notes how each of the fragments present in p_2 is related to the fragment to be edited. This is followed by the spatial context p_4 and the fragment to the edited p_5.

Oracle and Plan Iterations. Once all the nodes in the plan graph are marked as completed, an *iteration* of CodePlan is completed. As shown in Fig. 2, the oracle is invoked on the repository. If it flags any errors, the error locations and messages are used for seed changes for the next iteration and the planning resumes once again. If the oracle does not flag any errors, CodePlan terminates.

3 EXPERIMENTAL DESIGN

3.1 CodePlan Implementation

We provide a brief overview of the implementation components that constitute the core of CodePlan. We construct the Dependency Graph, crucial for representing code block relationships, by parsing code files using the "tree-sitter" library [31], which simplifies the identification of code blocks like classes, methods, import statements, and expressions. We establish edges within the Dependency Graph by tracing relationships within the AST, implementing custom logic for various relationships such as caller-callee, overrides-overridden, and more. In Python repositories, we utilize Jedi [10], a static analysis tool, to identify relationships like caller-callee, overrides-overridden, and base class-derived class. Our implementation integrates the `gpt-4-32k` LLM for code edits, providing it with structured input for enhanced quality and accuracy. We use temperature set to 0, top_p set to the default value of 1 and sample a single response for every call to the LLM. While our current implementation handles C# and Python repositories, it is easy to extend it to other programming languages due to the various abstractions and layered architecture of CodePlan.

3.2 Experimental Data

Tasks and Code Repositories. At present, there is no benchmark to evaluate repository-level coding tasks where examples consist of coordinated changes across a repository to satisfy some intent. We therefore construct a benchmark by selecting code repositories of varying complexities and sizes. This includes Internal C# Repositories (Int-1,2), large proprietary codebases requiring non-trivial migrations from legacy to modern logging frameworks. We also considered External Repositories from Public GitHub, focusing on Migration and Temporal Edits [85] tasks. For Migration, we selected C# repositories (Ext-1,2) having API or framework migrations, while for Temporal Edits, which involves series of code changes following initial edits, we selected Python repositories (T-1,2,3). We identified the GitHub repositories by searching for migration and multi-step temporal edit scenarios, and selected corresponding pull requests. As reported in Table 2, these repositories have between 4–168 files and 1.8K–20.4K lines of code.

Data Pre-processing. For each repository, we collect the ground truth before (*Source*) and after (*Target*) snapshots of the code from the pull requests. The pull requests contained code changes unrelated to the task. We either applied them to both Source and Target, or removed them from the Target. From the remaining changes, *seed changes* were identified through manual inspection. To prepare the Source for evaluation with both CodePlan and the baselines, we patch in the seed changes or prepare instructions for the LLM to carry them out. We observed that in contrast to the internal repositories, the external repositories did not have uniformity in the coding styles. Our initial experimentation revealed that this caused even the correct edits being flagged as differing from the ground truth edits. To mitigate this, we pre-process the Target repositories to ensure uniform coding practices (after the pre-processing). This may involve formatting changes such as standardizing whitespace, adding commas to lists or ordering imports as well as minor code changes such as enforcing common coding practices or removing code-edits unrelated to the task. Note that all methods are evaluated on the same Source repositories

Benchmark Statistics. We now discuss statistics of our benchmark to understand its scale and complexity (Table 2). The *number of files changed* range from 2–97. *Seed changes* are the number of

Table 2. Benchmark statistics.

Repositories	Migration				Temporal Edits		
	Int-1	Int-2	Ext-1	Ext-2	T-1	T-2	T-3
Number of files	91	168	55	341	21	137	4
Lines of code	8853	16476	8868	14305	3883	20413	1874
Number of files changed	47	97	21	23	2	2	3
Number of seed changes	41	63	42	50	2	1	1
Number of derived changes	110	375	22	68	8	3	10
Diff size b/w Source & Target (lines)	1744	4902	1024	154	104	15	39
Size of seed edits (lines)	242	242	379	340	76	4	1
Prompt template size (lines)	81	81	81	110	75	75	75
URL	-	-	[1]	[2]	[14]	[4]	[9]

initial edits (1–63 changes), considered as the starting point, and *derived changes* (3–375 changes) are the subsequent edits that follow the initial seed changes, which CodePlan is expected to automate. *Diff size b/w source and target (lines)* is the total number of lines (15–4.9K) in the file-wise diff between the Source and Target versions of the repositories. This tells us the size of the required code changes. Similarly, we report the *size of seed edits*. We used the same prompt template for C# migration across internal and public repositories (81 lines, as reported in *Prompt template size (lines)*) and another one (75 lines) for Python temporal edits.

3.3 Oracles and Baselines

Oracles. In our experiments, we rely on two specific oracles to evaluate the validity of our solutions. For C# migration tasks, passing C# Build tools [11] without errors serves as the oracle. In temporal edits scenarios, we use Pyright [12], a Python static checker, as the oracle. We also use the Pyright Strict mode which uses more checks compared to the default Pyright setting.

Oracle-Guided Repair Baselines. An alternative to our planning is to use the oracle to detect errors with each change. These approaches are reactive and involve attempting to fix errors identified by the oracles. We refer to them as *oracle-guided repair baselines*. For C# migration, we use Build-Repair, while for temporal edits, it's Pyright-Repair. The process includes applying an initial seed edit, detecting errors, analyzing error messages, and using an LLM for patching. However, oracle-guided repair may lack comprehensive change impact analysis, leading to potentially incomplete or incorrect fixes, especially in complex coding tasks. For fair comparison, we use the same contextualization method as CodePlan for the baselines.

Alternate Edit Models:. While CodePlan primarily leverages LLMs for localized code edits, it can also work with custom models like **Coeditor** [85]. Coeditor is designed for making an edit conditioned on prior temporal edits for Python code. We use Coeditor to evaluate whether CodePlan can work with different models and to perform a model ablation study. We also evaluate both the CodePlan pipeline along with the oracle-guided repair using **CodeLlama-34B** [74] which is an open source large language model.

3.4 Evaluation Metrics

We use two key metrics, Block Metrics and Edit Metrics, to assess how effectively CodePlan propagates changes throughout the code repository and the correctness of these changes.

Block Metrics. Block Metrics evaluate CodePlan's ability to identify code blocks in need of modification, including: *Matched Blocks:* Code blocks successfully identified for change; *Missed Blocks:* Code blocks that should have been modified but weren't; *Spurious Blocks:* Incorrectly edited blocks.

Edit Metrics:. Edit Metrics assess the correctness of CodePlan's modifications, including: *Levenshtein Distance:*, which measures edit distance between the Predicted and Target (ground truth)

Table 3. Comparison of CodePlan with baselines.

Dataset	Approach	Matched Blocks	Missed Blocks	Spurious Blocks	Diff BLEU	Levenshtein Distance	Validity Check
C# Migration Task on Internal (Proprietery) Repositories							
Int-1 (Logging)	CodePlan (Iter 1)	**151**	**0**	**0**	0.99	60	✗ (4) ≠
	CodePlan (Iter 2)	4	**0**	**0**	**1.00**	**0**	✓
	Build-Repair	82	69	13	0.81	6465	✗ (46) ≠
Int-2 (Logging)	CodePlan (Iter 1)	**438**	**0**	**0**	0.99	90	✗ (6) ≠
	CodePlan (Iter 2)	6	**0**	**0**	**1.00**	**0**	✓
	Build-Repair	337	101	25	0.66	7496	✗ (68) ≠
C# Migration Task on External (Public) Repositories							
Ext-1	CodePlan (Iter 1)	**64**	**0**	**0**	**0.86**	**2931**	✓
	Build-Repair	34	30	27	0.65	9145	✗ (40) ≠
Ext-2	CodePlan (Iter 1)	**38**	**8**	**0**	0.61	**1121**	✗ (13) ≠
	CodePlan (Iter 2)	2	0	6	**0.62**	1261	✗ (7) ≠
	Build-Repair	19	27	5	0.49	1379	✗ (11) ≠
Python Temporal Edit Task on External (Public) Repositories							
T-1	CodePlan (Iter 1)	**8**	**2**	0	**0.90**	**1044**	✗ (0) ≠
	Pyright-Repair	5	5	0	0.76	1089	✗ (0) ≠
	Pyright-Strict-Repair	**8**	**2**	0	**0.90**	**1045**	✗ (0) ≠
T-2	CodePlan (Iter 1)	**4**	**0**	0	**0.86**	**147**	✓
	Pyright-Repair	1	3	0	0.58	344	✗ (0) ≠
	Pyright-Strict-Repair	1	3	0	0.58	344	✗ (0) ≠
T-3	CodePlan (Iter 1)	**11**	**0**	0	**0.94**	**288**	✓
	Pyright-Repair	1	10	0	0.53	840	✗ (0) ≠
	Pyright-Strict-Repair	1	10	0	0.53	840	✗ (0) ≠

Repositories at the file level; and, *DiffBLEU*:, a modified BLEU [65] score focusing on comparing modified code sections while disregarding common code. Let Δ_{gt} and Δ_p respectively be diffs between the Source and Target repositories, and the Source and Predicted repositories. The BLEU score between Δ_{gt} and Δ_p gives us the DiffBLEU score. We use `git-diff` [5] to compute the diffs.

Validity Check. We say that a Predicted repository passes the *validity check* if the oracle (the build system for C# and Pyright for Python) does not detect any errors in it and we have a perfect match (modulo whitespace and formatting differences) with the ground truth Target repository.

4 RESULTS AND ANALYSIS

In this section, we present empirical results to answer the following research questions:

RQ1: How well is CodePlan able to localize and make the required changes to automate repository-level coding tasks compared to baselines?
RQ2: How important are temporal and spatial contexts to CodePlan's performance?
RQ3: What are the key differentiators that allow CodePlan to outperform baselines in solving complex coding tasks?
RQ4: How does CodePlan perform with different language models of code?
RQ5: How much does CodePlan cost in terms of API usage?

4.1 RQ1: How well is CodePlan able to localize and make the required changes to automate repository-level coding tasks compared to baselines?

This RQ investigates CodePlan's ability to automate complex repository-level coding tasks and compares it against the baselines. We consider two tasks from two programming languages (C# migration and Python temporal edits) and the baselines described in Section 3.

Table 3 reports the experimental results. Higher values of Matched Blocks and DiffBLEU, and lower values of Missed Blocks, Spurious Blocks, Levenshtein Distances are better. For each repository, different approaches are separated by a dashed line and the respective best values are highlighted in the bold font (except when all approaches have the same value). ✓ and ✗ respectively indicate if the Validity Check (Section 3.4) passes or fails, respectively. Against ✗, we also give the number of errors detected by the oracle in parentheses and indicate via ≠ that the output from the approach does not match the ground truth. In several cases in Python, even though the oracle (Pyright) does not flag any errors, the generated code does not match ground truth as indicated by "✗ (0) ≠" entries in the last column. This is because of the lack of sufficient type hints in the Python repositories to catch correctness requirements. In contrast, for the statically typed language C#, mismatch with ground truth is also reflected in non-zero build errors.

As seen in Table 3, CodePlan passes the validity checks in 5/7 repositories, whereas the baselines cannot pass them for any repository. This demonstrates the ability of CodePlan to automate complex repository-level coding tasks spanning many files (Table 2) and to work with multiple programming languages. We use the same spatial and temporal contextualization setup for CodePlan and the baselines (Section 3.3) and hence, the difference in the performance of CodePlan and the baselines stems from better accuracy of systematic planning in CodePlan compared to the oracle-guided repair in the baselines.

C# Migration Task on Internal (Proprietary) Repositories. CodePlan achieves 151 matched blocks for "Int-1 (Logging)" and 438 matched blocks for "Int-2 (Logging)," with zero missed blocks and spurious blocks. In contrast, Build-Repair falls behind with 82 matched blocks for "Int-1 (Logging)" and 337 for "Int-2 (Logging)," along with 69 and 101 missed blocks, respectively. Additionally, Build-Repair introduces 13 spurious blocks for "Int-1 (Logging)" and 25 for "Int-2 (Logging)." Notably, CodePlan does not introduce any build errors, unlike Build-Repair, which results in 46 and 68 errors for "Int-1 (Logging)" and "Int-2 (Logging)," respectively, which remain unresolved. This underscores CodePlan's precision and reliability for the C# Migration Task on internal repositories.

Multiple Iterations are essential due to the variability in LLM responses. After "Iter 1," CodePlan-generated code has 4 build errors in the "Int-1" dataset. "Iter 2" plays a crucial role in addressing these errors, re-engaging with the LLM, with the build error messages, to obtain more accurate responses and re-editing 4 blocks. Similar observations are made in the "Int-2" dataset.

CodePlan *versus Build-Repair.* A significant factor contributing to this performance difference is Build-Repair's reliance on "build error location" to indicate where code corrections are needed. However, build errors may not always align with the actual correction site, leading to misinterpretation. For instance, an error may manifest in a derived class's overridden function signature mismatch, but the fix is required in the base class's virtual function signature, causing Build-Repair to misinterpret the correction site. We list the key differentiating factors between CodePlan and baselines in RQ3 (Section 4.3).

C# Migration Task on External (Public) Repositories. In the comparison between CodePlan and the Build-Repair baseline on the "Ext-1" repository for the C# Migration Task, CodePlan successfully updated 64 code blocks. However, DiffBlue 0.86 (and Levenstein distance 2931) is due to the changes in code formatting and the order of method declarations in the predicted file. In contrast, Build-Repair missed 30 blocks and generated 40 build errors. Notably, Build-Repair failed to identify the correct blocks for updates because the reported build error locations are not always the right places to make edits. In contrast, CodePlan succeeded due to its change-impact analysis, accurately identifying the right locations to make the next edit. This underscores CodePlan's advanced planning abilities, ensuring precise and comprehensive repository-level code edits. In "Ext-2", the LLM did not perform a necessary type cast when using a library API, which was not

caught by CodePlan, resulting in missed blocks. Some of the resulting errors are fixed in "Iter-2". Build-Repair could not make progress in fixing the build errors completely and resulted in worse performance than CodePlan.

Python Temporal Edit Task on External (Public) Repositories. In the Python Temporal Edits task, CodePlan identified all derived edit locations across two repositories (T-2, T-3) but missed 2/10 locations in the third (T-1). In contrast, the Pyright-Repair baseline fails to identify any derived edits in two repositories (T-2, T-3). In T-2, Pyright doesn't flag errors for method call sites due to presence of a default parameter. In T-3, it misses edits required by changes in method behavior. CodePlan's change may-impact analysis handles these cases, whereas the oracle-guided repair baseline lacks such detection, focusing on fixing rule violations rather than propagating changes. Pyright in strict checking mode (Pyright-Strict-Repair) improves results but matches CodePlan only in one repository (T-1). CodePlan consistently outperforms in DiffBLEU score and has lower Levenshtein Distance, although not always achieving perfect 1.0 and 0 values due to slight differences in LLM edits and ground truth. We provide more details in the supplementary material.

Shortcomings of CodePlan. While CodePlan is able to complete the required edits in 5 out of 7 scenarios, there are cases where it falls short. These can be broadly categorized as (1) incorrect edits from the language model and (2) incomplete static analysis. As an example of (1), consider Ext-2, where the model fails to make a correct edit, missing out on a necessary type cast. While the build tool is able to catch some errors caused by this (which CodePlan fixes in subsequent iterations), the respository still ends up with some errors. In cases of (2) incomplete static analysis, CodePlan can end up missing blocks to edit due to missing edges in the Dependency Graph. For example, in T-1, the missed edit site is a function which stores a dictionary where the values are classes. This dictionary is indexed into and the resulting class is instantiated and used. Since a prior edits had been made to the definitions of the classes in the dictionary, we needed to edit how objects of these classes were being used in this function. However, since our static analysis did not identify that these classes were being used, this function was not considered during change propagation. Note that the impact of both of these shortcomings can be reduced with better models and analysis tools.

4.2 RQ2: How important are temporal and spatial contexts to CodePlan's performance?

We performed this ablation on one internal and one external repository for C# migration and all Python repositories (which are smaller).

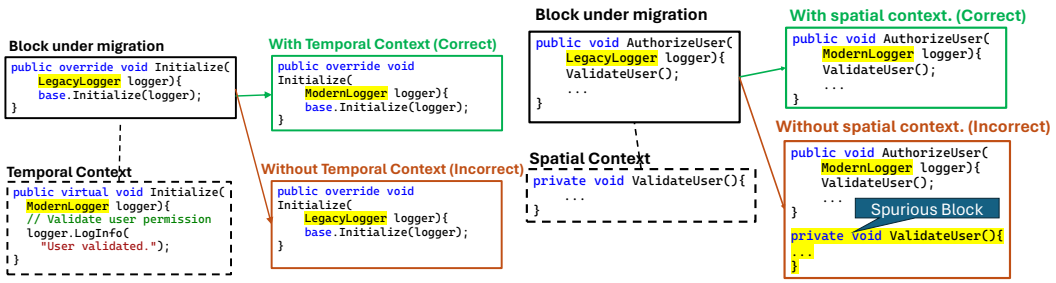
Importance of Temporal Context. As observed in Table 4, when temporal contexts are not considered, there is a noticeable increase in missed blocks. This increase is attributed to the LLM not making necessary changes to certain code blocks due to its inability to comprehend the need for those modifications in the absence of temporal context.

An illustrative example in Fig. 5a exemplifies this issue. In this scenario, a correction is required in the derived class's overriding method based on changes to the virtual method's signature in the base class. However, the LLM, lacking temporal context, does not possess information about the base class's method, leading it to believe that no changes are necessary to the derived class method.

Importance of Spatial Context. It's crucial to highlight another significant observation: the increase in the count of spurious blocks when spatial context is insufficient. This phenomenon occurs because, in the absence of adequate spatial context, the LLM may incorrectly attempt to re-create blocks that exist in the code but are not supplied in the prompt, leading to the generation of spurious code blocks. An illustrative example in Fig. 5b demonstrates this issue. In this scenario, the task is to modify the `AuthorizeUser` method by migrating the logging calls from an old logging framework to a new one. However, due to the lack of spatial context that would specify the existence of the `ValidateUser` method, the LLM attempts to unnecessarily create this method as well.

Table 4. Ablation study with and without temporal/spatial context. For Python Temporal Edit task (T-1,2,3), temporal context is the necessary part of input and hence, only spatial context is ablated.

Dataset	Approach	Matched Blocks	Missed Blocks	Spurious Blocks	Diff BLEU	Levenshtein Distance	Validity Check
Int-1	CodePlan	151	0	0	1.00	0	✓
	– Temporal Context	135	16	32	0.63	3892	✗ (61) ≠
	– Spatial Context	134	17	51	0.61	4161	✗ (65) ≠
	– Temporal and Spatial	121	30	54	0.51	4524	✗ (69) ≠
Ext-1	CodePlan	65	0	0	0.86	2931	✓
	– Temporal Context	62	3	2	0.74	1014	✗ (8) ≠
	– Spatial Context	62	3	2	0.74	1014	✗ (8) ≠
	– Temporal and Spatial	61	4	2	0.71	1036	✗ (9) ≠
T-1	CodePlan	8	2	0	0.90	1044	✗ (0) ≠
	– Spatial Context	8	2	0	0.89	1266	✗ (0) ≠
T-2	CodePlan	4	0	0	0.86	147	✓
	– Spatial Context	4	0	0	0.76	443	✓
T-3	CodePlan	11	0	0	0.94	288	✓
	– Spatial Context	11	0	0	0.92	325	✓



(a) Temporal (b) Spatial

Fig. 5. Illustration of the importance of (a) temporal and (b) spatial context.

Summary. We also ablate both spatial and temporal contexts for C# repositories. The Python task requires temporal context as input by definition, hence, we only ablate the spatial context. The general observation is that not providing the necessary (spatial or temporal) context results in degraded output, resulting in increase in missing or spurious blocks, or decrease in match with ground truth (as seen in decrease in DiffBLEU scores).

4.3 RQ3: What are the key differentiators that allow CodePlan to outperform baselines in solving complex coding tasks?

CodePlan's strong performance in complex coding tasks is due to its advanced features, especially its incremental and change may-impact analysis. These features distinguish it from baseline methods like Build-Repair, which prioritize syntactic correctness but overlook contextual details and change propagation. For instance, in a scenario from repository Ext-1, where CodePlan is tasked with migrating `Console.WriteLine` to `ITestOutputHelper.WriteLine`, it effectively executes a series of changes, as shown in steps 1-4 in Fig. 6, while Build-Repair fails to perform steps 2-4. Please refer to the descriptions in the inset in Fig. 6 for further details.

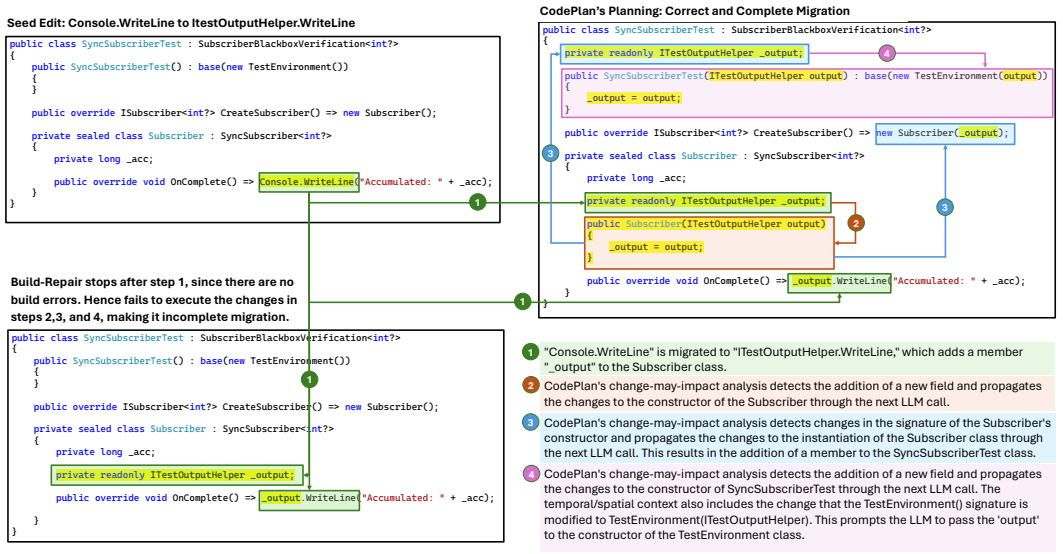


Fig. 6. Illustration of CodePlan's plan execution versus incomplete changes made by the Build-Repair baseline.

We have performed an extensive qualitative evaluation of CodePlan and all baselines. Due to space limitations, we state our observations briefly below. Please refer to the supplementary material for detailed discussion on these points.

- Incremental analysis preserves dependency graph relationships across edits.
- Incremental analysis facilitates joint extraction of spatial and temporal context.
- Change may-impact analysis propagates subtle behavioral changes.
- Change may-impact analysis preserves cause-effect relationships.
- Incremental static analysis is lightweight and deployable.

4.4 RQ4: How does CodePlan perform with different language models of code?

CodePlan provides a framework for repository-level tasks with the choice of the language model being flexible. To demonstrate this flexibility and study the behavior of CodePlan with language models other than `gpt-4-32k`, we experiment with both CodeLlama [74] and Coeditor [85] and present the results in Table 5. Since Coeditor is a Python-specific model, we perform this experiment on the Python Temporal Edits task. Comparing CodePlan (with the GPT model) and Coeditor-CodePlan, we see that they perform similarly on T-1 but slightly differ on T-2 and T-3, with Coeditor-CodePlan missing one edit site in each. In both cases, Coeditor misses adding an argument to a method being edited, thus missing out on editing the callers of that method. We also observe lower DiffBLEU scores and higher Levenshtein Distance (L.D.) in T-2 and T-3 for Coeditor-CodePlan compared to CodePlan. This may be attributed to `gpt-4-32k`'s superior contextual understanding, seen in the better alignment with ground truth edits compared to Coeditor. We also replaced the GPT model with Coeditor in the Pyright-Repair and Pyright-Strict-Repair baselines, giving use Coeditor-Pyright-Repair and Coeditor-Pyright-Strict-Repair baselines. The identical results indicate that the inability of the oracle-guided repair baselines in identifying required changes is orthogonal to the choice of the model. Similar observations are made in the case of CodeLlama. While CodeLlama-CodePlan is able to edit all required blocks in T-2, when editing one function, CodeLlama opts to add a parameter which was not added in the ground truth, leading to the system editing 8 spurious blocks. We also observe that CodeLlama sometimes does not make the required edit at all (as is the case in T-1 and T-3), leading to missed blocks.

Table 5. Performance CodePlan with different language models of code.

Dataset	Approach	Matched Blocks	Missed Blocks	Spurious Blocks	Diff BLEU	Levenshtein Distance	Validity Check
Python Temporal Edit Task on External (Public) Repositories							
T-1	CodePlan	**8**	**2**	0	**0.90**	**1044**	✗ (0) ≠
	Pyright-Repair	5	5	0	0.76	1089	✗ (0) ≠
	Pyright-Strict-Repair	**8**	**2**	0	**0.90**	**1045**	✗ (0) ≠
	Coeditor-CodePlan	**8**	**2**	0	**0.90**	1160	✗ (0) ≠
	Coeditor-Pyright-Repair	5	5	0	0.66	1206	✗ (0) ≠
	Coeditor-Pyright-Strict-Repair	**8**	**2**	0	0.83	1106	✗ (6) ≠
	CodeLlama-CodePlan	6	4	0	0.68	1112	✗ (3) ≠
	CodeLlama-Pyright-Repair	5	5	0	0.65	1145	✗ (1) ≠
	CodeLlama-Pyright-Strict-Repair	6	4	0	0.73	1163	✗ (3) ≠
T-2	CodePlan	**4**	**0**	0	**0.86**	**147**	✓
	Pyright-Repair	1	3	0	0.58	344	✗ (0) ≠
	Pyright-Strict-Repair	1	3	0	0.58	344	✗ (0) ≠
	Coeditor-CodePlan	2	2	0	0.82	254	✗ (0) ≠
	Coeditor-Pyright-Repair	1	3	0	0.58	344	✗ (0) ≠
	Coeditor-Pyright-Strict-Repair	1	3	0	0.58	344	✗ (0) ≠
	CodeLlama-CodePlan	**4**	**0**	8	0.69	326	✗ (0) ≠
	CodeLlama-Pyright-Repair	1	3	0	0.58	344	✗ (0) ≠
	CodeLlama-Pyright-Strict-Repair	1	3	0	0.58	344	✗ (0) ≠
T-3	CodePlan	**11**	**0**	0	**0.94**	**288**	✓
	Pyright-Repair	1	10	0	0.53	840	✗ (0) ≠
	Pyright-Strict-Repair	1	10	0	0.53	840	✗ (0) ≠
	Coeditor-CodePlan	10	1	0	0.76	759	✗ (0) ≠
	Coeditor-Pyright-Repair	1	10	0	0.53	840	✗ (0) ≠
	Coeditor-Pyright-Strict-Repair	1	10	0	0.53	840	✗ (0) ≠
	CodeLlama-CodePlan	1	10	0	0.53	840	✗ (0) ≠
	CodeLlama-Pyright-Repair	1	10	0	0.53	840	✗ (0) ≠
	CodeLlama-Pyright-Strict-Repair	1	10	0	0.53	840	✗ (0) ≠

4.5 RQ5: How much does CodePlan cost in terms of API usage?

The language model used to make edits makes up a key component of CodePlan. While open source language models are available which can be used for free to generate such edits, we understand that there might be a cost associated with performing inferences on cloud-hosted models such as `gpt-4`. To better understand the scale of these costs, we analyzed CodePlan logs to estimate the number of API calls, tokens used in these API calls and charge incurred per experiment (presented in Table 6). These estimates are based on pricing from the OpenAI website at the time of writing ($0.06 / 1K tokens for input and $0.12 / 1K tokens for output). We can see the API cost per task ranges from $0.3 for T-2 (with ground truth diff of 15 lines) to $32 for Int-1 (with ground truth diff of 1.7K lines) – a very reasonable range when compared to the developer time and effort spent when making these edits manually.

Table 6. API usage (tokens) and cost (USD).

	Calls	Prompt	Response	Total
Int-1	172	399K **$24**	66K **$8**	465K **$32**
Int-2	71	99K **$6**	19K **$2**	118K **$8**
Ext-1	108	100K **$6**	17K **$2**	118K **$8**
Ext-2	80	79K **$5**	26K **$3**	105K **$8**
T-1	7	6.5K **$0.4**	0.7K **$0.1**	7.2K **$0.5**
T-2	3	2.7K **$0.2**	917 **$0.1**	3.6K **$0.3**
T-3	10	23.9K **$1.4**	5K **$0.6**	28.9K **$2**

5 LIMITATIONS AND THREATS TO VALIDITY

CodePlan relies on high-quality dependency analysis, which works well in statically typed languages like C# and Java but can be challenging in dynamically typed languages like Python or JavaScript without type hints due to their dynamic nature.

Our current CodePlan implementation mainly deals with code block relations through static analysis. However, real-world software systems have dynamic dependencies, like data flows, complex dispatching, and execution dependencies, and include various artifacts beyond code files. Addressing these dynamic dependencies and software artifacts is a priority for our future work.

CodePlan edits one code block at a time, which might not be the most efficient approach in all cases. Also, LLMs can make errors while editing code. Our ablations show that CodePlan's spatial and temporal context helps avoid such errors considerably. Besides, instead of blindly trusting the changes made by the LLM, CodePlan employs an oracle to validate the changes and initiates further iterations if the changes are found unsatisfactory. This oracle-in-the-loop strategy helped us get to the desired, error-free edits in multiple C# migration cases. We want to explore techniques to exploit feedback from oracles to improve reliability of repository-wide changes.

We chose multiple repositories for two challenging tasks (migration and temporal edits) in two languages (C# and Python) to assess CodePlan's generality. These tasks and repositories represent real-world scenarios. However, due to limited access to the LLM, our evaluation is confined to the current experiments. There is a potential concern that the edits we are asking the model to perform might have been part of the LLM's training set. To address this, we conducted experiments on two proprietary internal C# repositories whose source code the LLM didn't encounter during training. Moreover, except for Ext-1, our tasks use GitHub pull requests created after September 2021, the LLM's training data cutoff date, for which the LLM may have seen an older variant of the codebase, but not the edit that we are requesting. We intentionally included Ext-1 before this date to test if the model could perform better, but our baseline and ablation results indicate that it couldn't make the desired edits without appropriate context. We aim to expand our experimental results to include more repositories in the future. Practically, rate limits on cloud-hosted language models may pose issues around usability, however as demonstrated in experiments CodePlan also supports using open source language models for which rate limits are not a limitation.

Although our current methodology employs zero-shot prompting, there exists potential to include few-shot examples [30], Chain of Thought (CoT) [87], and other techniques, which can improve the performance of CodePlan further.

6 RELATED WORK

LLMs for Coding Tasks. A multitude of LLMs [16, 24, 27, 30, 34, 36, 41, 64, 81, 83, 84, 89] have been trained on large-scale corpora of source code and natural language text. These have been used to accomplish a variety of coding tasks. A few examples of their use include program synthesis [57, 63], program repair [17, 50, 88], vulnerability patching [68], inferring program invariants [70], test generation [77] and multi-task evaluation [80]. However, these investigations are performed on curated examples that are extracted from their repositories and are meant to be accomplished with independent invocations of the LLM. We consider a different class of tasks posed at the scale of code repositories, where an LLM is called multiple times on different examples which are inter-dependent. We monitor the results of each LLM invocation within the repository-wide context to identify future code change obligations to get the repository to a valid state, e.g., where the repository is free of build or runtime errors.

Neuro-symbolic frameworks. Many works propose combining learning based methods such as neural networks with symbolic techniques such as search for synthesizing programs. [66] performs neural search over programs, [26] trains models to predict properties of programs from given input-output pairs while [82] combines models with deductive search. However such approaches are limited to generating small code snippets, require input-output examples and often operate over domain specific languages (DSLs). CodePlan on the other hand uses languages models together with planning and static analysis for making multiple edits across real world repositories.

Automated Planning. Automated planning [43, 75] is a well-studied topic in AI. Online planning [75] is used when the effect of actions is not known and the state-space cannot be enumerated *a priori*. It requires monitoring the actions and plan extension. In our case, the edit actions are carried out by an LLM whose results cannot be predicted before-hand and the state-space is unbounded. As a consequence, our adaptive planning is an online algorithm where we monitor the actions and extend the plan through static analysis. In orthogonal directions, [49] uses an LLM to derive a plan given a natural language intent before generating code to solve complex coding problems and [95] performs lookahead planning (tree search) to guide token-level decoding of code LMs. Planning in our work is based on analyzing dependency relations and changes to them as an LLM makes changes to a code repository.

Analysis of Code Changes. Static analysis is used for ensuring software quality. It is expensive to recompute the analysis results every time the code undergoes changes. The field of incremental program analysis offers techniques to recompute only the analysis results impacted by the change. Specialized algorithms have been developed for dataflow analysis [23, 76], pointer analysis [93], symbolic execution [71], bug detection [59] and type analysis [33]. Program differencing [21, 53, 55] and change impact analysis [22, 48] determine the differences in two program versions and the effect of a change on the rest of the program. The impact of changes has been studied for regression testing [73], analyzing refactorings [39] and assisting in code review [19, 42]. We analyze the code generated by an LLM and incrementally update the syntactic (e.g., parent-child) and dependency (e.g., caller-callee) relations. We further analyze the likely impact of those changes on related code blocks and create change obligations to be discharged by the LLM.

Spatial and Temporal Contextualization. As discussed in the Introduction, LLMs benefit from relevant context derived from other files in the repository and from past edits. For example, RLPG [79] learns to select relevant parts of the repository to include as input for code generation while Coeditor [85] trains a language model to generate code edits conditioned on prior related edits made across the repository. We not only provide both these pieces of information to the LLM by tracking the code changes and dependency relations but go a step further by inferring where and how the current edit may affect the rest of the repository.

Learning Edit Patterns. Many approaches have been developed to learn edit patterns from past edits or commits in the form of rewrite rules [37], bug fixes [20, 25], type changes [52], API migrations [56, 91] and neural representations of edits [92]. Approaches such as [60] and [61] synthesize context-aware edit scripts from user-provided examples and apply them in new contexts. Other approaches observe the user actions in an IDE to automate repetitive edits [62] and temporally-related edit sequences [96]. We do not aim to learn edit patterns and we do not assume similarities between edits. Our focus is to identify effects of code changes made by an LLM and to guide the LLM towards additional changes that become necessary.

7 CONCLUSIONS AND FUTURE WORK

In this paper, we introduced CodePlan, a neuro-symbolic framework for handling complex repository-level coding tasks involving extensive code changes across interdependent files in large codebases. CodePlan employs incremental dependency analysis, change may-impact analysis, and adaptive planning to coordinate multi-step code edits using large language models. Our evaluation on various code repositories in C# and Python demonstrated that CodePlan surpasses baseline methods in accuracy. It shows great promise for automating repository-level coding tasks, but there's room for future improvements. We plan to extend its applicability to more programming languages and explore enhancements to its editing strategy and analysis. Additionally, we aim to conduct large-scale experiments to further refine CodePlan's effectiveness across diverse coding tasks.

REFERENCES

[1] 2020. Reactive Streams TCK. https://github.com/reactive-streams/reactive-streams-dotnet/tree/master/src/tck.
[2] 2022. das-qna-api. https://github.com/SkillsFundingAgency/das-qna-api.
[3] 2023. Amazon Code Whisperer - AI Code Generator. https://aws.amazon.com/codewhisperer/.
[4] 2023. audiocraft. https://github.com/facebookresearch/audiocraft.
[5] 2023. git-diff. https://git-scm.com/docs/git-diff.
[6] 2023. GitHub Copilot chat for Visual Studio 2022. https://devblogs.microsoft.com/visualstudio/github-copilot-chat-for-visual-studio-2022/.
[7] 2023. GitHub Copilot: Your AI pair programmer. https://github.com/features/copilot.
[8] 2023. GPT-4 32K. https://platform.openai.com/docs/models/gpt-4.
[9] 2023. JARVIS. https://github.com/microsoft/JARVIS.
[10] 2023. Jedi. https://github.com/davidhalter/jedi.
[11] 2023. MS-Build. https://learn.microsoft.com/en-us/visualstudio/msbuild/msbuild.
[12] 2023. Pyright. https://github.com/microsoft/pyright.
[13] 2023. Replit. https://replit.com/.
[14] 2023. whisper. https://github.com/openai/whisper.
[15] Lakshya A Agrawal, Aditya Kanade, Navin Goyal, Shuvendu K. Lahiri, and Sriram K. Rajamani. 2023. Guiding Language Models of Code with Global Context using Monitors. arXiv:2306.10763 [cs.CL]
[16] Wasi Uddin Ahmad, Saikat Chakraborty, Baishakhi Ray, and Kai-Wei Chang. 2021. Unified Pre-training for Program Understanding and Generation. arXiv:2103.06333 [cs.CL]
[17] Toufique Ahmed and Premkumar Devanbu. 2023. Better patching using LLM prompting, via Self-Consistency. arXiv:2306.00108 [cs.SE]
[18] Alfred V Aho, Ravi Sethi, Jeffrey D Ullman, et al. 2007. *Compilers: principles, techniques, and tools*. Vol. 2. Addison-wesley Reading.
[19] Everton L. G. Alves, Myoungkyu Song, and Miryung Kim. 2014. RefDistiller: A Refactoring Aware Code Review Tool for Inspecting Manual Refactoring Edits. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering* (Hong Kong, China) *(FSE 2014)*. Association for Computing Machinery, New York, NY, USA, 751–754. https://doi.org/10.1145/2635868.2661674
[20] Jesper Andersen and Julia L Lawall. 2010. Generic patch inference. *Automated software engineering* 17 (2010), 119–148.
[21] Taweesup Apiwattanapong, Alessandro Orso, and Mary Jean Harrold. 2004. A differencing algorithm for object-oriented programs. In *Proceedings. 19th International Conference on Automated Software Engineering, 2004.* IEEE, 2–13.
[22] RS Arnold and SA Bohner. 1996. An introduction to software change impact analysis. *Software Change Impact Analysis* (1996), 1–26.
[23] Steven Arzt and Eric Bodden. 2014. Reviser: efficiently updating IDE-/IFDS-based data-flow analyses in response to incremental program changes. In *Proceedings of the 36th International Conference on Software Engineering*. 288–298.
[24] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. 2021. Program Synthesis with Large Language Models. http://arxiv.org/abs/2108.07732 arXiv:2108.07732 [cs].
[25] Johannes Bader, Andrew Scott, Michael Pradel, and Satish Chandra. 2019. Getafix: Learning to Fix Bugs Automatically. *Proc. ACM Program. Lang.* 3, OOPSLA, Article 159 (Oct. 2019), 27 pages. https://doi.org/10.1145/3360585
[26] Matej Balog, Alexander L. Gaunt, Marc Brockschmidt, Sebastian Nowozin, and Daniel Tarlow. 2016. DeepCoder: Learning to Write Programs. *ArXiv* abs/1611.01989 (2016). https://api.semanticscholar.org/CorpusID:2906360
[27] Sid Black, Stella Biderman, Eric Hallahan, Quentin Anthony, Leo Gao, Laurence Golding, Horace He, Connor Leahy, Kyle McDonell, Jason Phang, and others. 2022. Gpt-neox-20b: An open-source autoregressive language model. *arXiv preprint arXiv:2204.06745* (2022).
[28] Bruno Blanchet. 2003. Escape analysis for JavaTM: Theory and practice. *ACM Transactions on Programming Languages and Systems (TOPLAS)* 25, 6 (2003), 713–775.
[29] Shaked Brody, Uri Alon, and Eran Yahav. 2020. A structural model for contextual code changes. 4, OOPSLA (Nov. 2020). https://doi.org/10.1145/3428283 Publisher Copyright: © 2020 Owner/Author..
[30] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.
[31] Max Brunsfeld, Andrew Hlynskyi, Patrick Thomson, Josh Vera, Phil Turnbull, Timothy Clem, Douglas Creager, Andrew Helwer, Rob Rix, Hendrik Van Antwerpen, Michael Davis, , Ika, Tun-Anh Nguyn, Stafford Brunk, Niranjan Hasabnis, Bfredl, Mingkai Dong, Matt Massicotte, Jonathan Arnett, Vladimir Panteleev, Steven Kalt, Kolja Lampe, Alex Pinkus, Mark Schmitz, Matthew Krupcale, Narpfel, Santos Gallegos, Vicent Martí, and , Edgar. 2023. tree-sitter/tree-sitter: v0.20.8. https://doi.org/10.5281/ZENODO.4619183

[32] Alan Bundy. 1988. The use of explicit plans to guide inductive proofs. In *9th International Conference on Automated Deduction: Argonne, Illinois, USA, May 23–26, 1988 Proceedings 9*. Springer, 111–120.

[33] Matteo Busi, Pierpaolo Degano, and Letterio Galletta. 2019. Using standard typing algorithms incrementally. In *NASA Formal Methods: 11th International Symposium, NFM 2019, Houston, TX, USA, May 7–9, 2019, Proceedings 11*. Springer, 106–122.

[34] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, and others. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).

[35] Jong-Deok Choi, Manish Gupta, Mauricio Serrano, Vugranam C Sreedhar, and Sam Midkiff. 1999. Escape analysis for Java. *Acm Sigplan Notices* 34, 10 (1999), 1–19.

[36] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. 2022. Palm: Scaling language modeling with pathways. *arXiv preprint arXiv:2204.02311* (2022).

[37] Reudismam Rolim de Sousa, Gustavo Soares, Rohit Gheyi, Titus Barik, and Loris D'Antoni. 2021. Learning Quick Fixes from Code Repositories. In *SBES '21: 35th Brazilian Symposium on Software Engineering, Joinville, Santa Catarina, Brazil, 27 September 2021 - 1 October 2021*, Cristiano D. Vasconcellos, Karina Girardi Roggia, Vanessa Collere, and Paulo Bousfield (Eds.). ACM, 74–83. https://doi.org/10.1145/3474624.3474650

[38] Jeffrey Dean, David Grove, and Craig Chambers. 1995. Optimization of object-oriented programs using static class hierarchy analysis. In *ECOOP'95—Object-Oriented Programming, 9th European Conference, Åarhus, Denmark, August 7–11, 1995 9*. Springer, 77–101.

[39] Danny Dig, Can Comertoglu, Darko Marinov, and Ralph Johnson. 2006. Automated detection of refactorings in evolving components. In *ECOOP 2006–Object-Oriented Programming: 20th European Conference, Nantes, France, July 3-7, 2006. Proceedings 20*. Springer, 404–428.

[40] Yangruibo Ding, Zijian Wang, Wasi Uddin Ahmad, Murali Krishna Ramanathan, Ramesh Nallapati, Parminder Bhatia, Dan Roth, and Bing Xiang. 2022. CoCoMIC: Code Completion By Jointly Modeling In-file and Cross-file Context. http://arxiv.org/abs/2212.10007 arXiv:2212.10007 [cs].

[41] Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Wen-tau Yih, Luke Zettlemoyer, and Mike Lewis. 2022. Incoder: A generative model for code infilling and synthesis. *arXiv preprint arXiv:2204.05999* (2022).

[42] Xi Ge, Saurabh Sarkar, Jim Witschey, and Emerson Murphy-Hill. 2017. Refactoring-aware code review. In *2017 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. IEEE, 71–79.

[43] Malik Ghallab, Dana Nau, and Paolo Traverso. 2004. *Automated Planning: theory and practice*. Elsevier.

[44] David González, Joshué Pérez, Vicente Milanés, and Fawzi Nashashibi. 2015. A review of motion planning techniques for automated vehicles. *IEEE Transactions on intelligent transportation systems* 17, 4 (2015), 1135–1145.

[45] Priyanshu Gupta, Avishree Khare, Yasharth Bajpai, Saikat Chakraborty, Sumit Gulwani, Aditya Kanade, Arjun Radhakrishna, Gustavo Soares, and Ashish Tiwari. 2023. GrACE: Generation using Associated Code Edits. arXiv:2305.14129 [cs.SE]

[46] Abram Hindle, Earl T Barr, Mark Gabel, Zhendong Su, and Premkumar Devanbu. 2016. On the naturalness of software. *Commun. ACM* 59, 5 (2016), 122–131.

[47] Pascal Hitzler and Md Kamruzzaman Sarker. 2022. Neuro-symbolic artificial intelligence: The state of the art. (2022).

[48] Mohammad-Amin Jashki, Reza Zafarani, and Ebrahim Bagheri. 2008. Towards a more efficient static software change impact analysis method. In *Proceedings of the 8th ACM SIGPLAN-SIGSOFT workshop on Program analysis for software tools and engineering*. 84–90.

[49] Xue Jiang, Yihong Dong, Lecheng Wang, Qiwei Shang, and Ge Li. 2023. Self-planning Code Generation with Large Language Model. arXiv:2303.06689 [cs.SE]

[50] Matthew Jin, Syed Shahriar, Michele Tufano, Xin Shi, Shuai Lu, Neel Sundaresan, and Alexey Svyatkovskiy. 2023. InferFix: End-to-End Program Repair with LLMs. *arXiv preprint arXiv:2303.07263* (2023).

[51] Erez Karpas and Daniele Magazzeni. 2020. Automated planning for robotics. *Annual Review of Control, Robotics, and Autonomous Systems* 3 (2020), 417–439.

[52] Ameya Ketkar, Oleg Smirnov, Nikolaos Tsantalis, Danny Dig, and Timofey Bryksin. 2022. Inferring and applying type changes. In *Proceedings of the 44th International Conference on Software Engineering*. 1206–1218.

[53] Miryung Kim, David Notkin, Dan Grossman, and Gary Wilson. 2012. Identifying and summarizing systematic code changes via rule inference. *IEEE Transactions on Software Engineering* 39, 1 (2012), 45–62.

[54] Steven M La Valle. 2011. Motion planning. *IEEE Robotics & Automation Magazine* 18, 2 (2011), 108–118.

[55] Shuvendu K Lahiri, Chris Hawblitzel, Ming Kawaguchi, and Henrique Rebêlo. 2012. Symdiff: A language-agnostic semantic diff tool for imperative programs. In *Computer Aided Verification: 24th International Conference, CAV 2012, Berkeley, CA, USA, July 7-13, 2012 Proceedings 24*. Springer, 712–717.

[56] Maxime Lamothe, Weiyi Shang, and Tse-Hsun Peter Chen. 2020. A3: Assisting android api migrations using code examples. *IEEE Transactions on Software Engineering* 48, 2 (2020), 417–431.

[57] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d'Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. 2022. Competition-level code generation with AlphaCode. *Science* 378, 6624 (2022), 1092–1097. https://doi.org/10.1126/science.abq1158 _eprint: https://www.science.org/doi/pdf/10.1126/science.abq1158.

[58] Tianyang Liu, Canwen Xu, and Julian McAuley. 2023. RepoBench: Benchmarking Repository-Level Code Auto-Completion Systems. arXiv:2306.03091 [cs.CL]

[59] Scott McPeak, Charles-Henri Gros, and Murali Krishna Ramanathan. 2013. Scalable and incremental software bug detection. In *Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering*. 554–564.

[60] Na Meng, Miryung Kim, and Kathryn S McKinley. 2011. Sydit: Creating and applying a program transformation from an example. In *Proceedings of the 19th ACM SIGSOFT symposium and the 13th European conference on Foundations of software engineering*. 440–443.

[61] Na Meng, Miryung Kim, and Kathryn S McKinley. 2013. LASE: locating and applying systematic edits by learning from examples. In *2013 35th International Conference on Software Engineering (ICSE)*. IEEE, 502–511.

[62] Anders Miltner, Sumit Gulwani, Vu Le, Alan Leung, Arjun Radhakrishna, Gustavo Soares, Ashish Tiwari, and Abhishek Udupa. 2019. On the fly synthesis of edit suggestions. *Proceedings of the ACM on Programming Languages* 3, OOPSLA (2019), 1–29.

[63] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. 2023. CodeGen: An Open Large Language Model for Code with Multi-Turn Program Synthesis. In *The Eleventh International Conference on Learning Representations*. https://openreview.net/forum?id=iaYcJKpY2B_

[64] OpenAI. 2023. GPT-4 Technical Report. arXiv:2303.08774 [cs.CL]

[65] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. Bleu: a method for automatic evaluation of machine translation. In *Proceedings of the 40th annual meeting of the Association for Computational Linguistics*. 311–318.

[66] Emilio Parisotto, Abdel rahman Mohamed, Rishabh Singh, Lihong Li, Dengyong Zhou, and Pushmeet Kohli. 2016. Neuro-Symbolic Program Synthesis. *ArXiv* abs/1611.01855 (2016). https://api.semanticscholar.org/CorpusID:15904815

[67] Pardis Pashakhanloo, Aaditya Naik, Yuepeng Wang, Hanjun Dai, Petros Maniatis, and Mayur Naik. 2022. Codetrek: Flexible modeling of code using an extensible relational representation. (2022).

[68] Hammond Pearce, Benjamin Tan, Baleegh Ahmad, Ramesh Karri, and Brendan Dolan-Gavitt. 2022. Examining Zero-Shot Vulnerability Repair with Large Language Models. In *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE Computer Society, 1–18.

[69] Hengzhi Pei, Jinman Zhao, Leonard Lausen, Sheng Zha, and George Karypis. 2023. Better context makes better code language models: A case study on function call argument completion. In *AAAI*. https://www.amazon.science/publications/better-context-makes-better-code-language-models-a-case-study-on-function-call-argument-completion

[70] Kexin Pei, David Bieber, Kensen Shi, Charles Sutton, and Pengcheng Yin. 2023. Can Large Language Models Reason about Program Invariants? (2023).

[71] Suzette Person, Guowei Yang, Neha Rungta, and Sarfraz Khurshid. 2011. Directed incremental symbolic execution. *Acm Sigplan Notices* 46, 6 (2011), 504–515.

[72] Machel Reid and Graham Neubig. 2022. Learning to Model Editing Processes. https://doi.org/10.48550/ARXIV.2205.12374

[73] Xiaoxia Ren, Fenil Shah, Frank Tip, Barbara G Ryder, and Ophelia Chesley. 2004. Chianti: a tool for change impact analysis of java programs. In *Proceedings of the 19th annual ACM SIGPLAN conference on Object-oriented programming, systems, languages, and applications*. 432–448.

[74] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, I. Evtimov, Joanna Bitton, Manish P Bhatt, Cristian Cantón Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre D'efossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. 2023. Code Llama: Open Foundation Models for Code. *ArXiv* abs/2308.12950 (2023). https://api.semanticscholar.org/CorpusID:261100919

[75] Stuart J Russell. 2010. *Artificial intelligence a modern approach*. Pearson Education, Inc.

[76] Barbara G Ryder. 1983. Incremental data flow analysis. In *Proceedings of the 10th ACM SIGACT-SIGPLAN symposium on Principles of programming languages*. 167–176.

[77] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. 2023. Adaptive test generation using a large language model. *arXiv preprint arXiv:2302.06527* (2023).

[78] Disha Shrivastava, Denis Kocetkov, Harm de Vries, Dzmitry Bahdanau, and Torsten Scholak. 2023. RepoFusion: Training Code Models to Understand Your Repository. arXiv:2306.10998 [cs.LG]

[79] Disha Shrivastava, Hugo Larochelle, and Daniel Tarlow. 2022. Repository-level prompt generation for large language models of code. *arXiv preprint arXiv:2206.12839* (2022).

[80] Haoye Tian, Weiqi Lu, Tsz On Li, Xunzhu Tang, Shing-Chi Cheung, Jacques Klein, and Tegawendé F. Bissyandé. 2023. Is ChatGPT the Ultimate Programming Assistant – How far is it? arXiv:2304.11938 [cs.SE]

[81] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. arXiv:2307.09288 [cs.CL]

[82] Ashwin J. Vijayakumar, Abhishek Mohta, Oleksandr Polozov, Dhruv Batra, Prateek Jain, and Sumit Gulwani. 2018. Neural-Guided Deductive Search for Real-Time Program Synthesis from Examples. *ArXiv* abs/1804.01186 (2018). https://api.semanticscholar.org/CorpusID:4606753

[83] Ben Wang and Aran Komatsuzaki. 2021. GPT-J-6B: A 6 billion parameter autoregressive language model.

[84] Yue Wang, Weishi Wang, Shafiq R. Joty, and Steven C. H. Hoi. 2021. CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation. *ArXiv* abs/2109.00859 (2021).

[85] Jiayi Wei, Greg Durrett, and Isil Dillig. 2023. Coeditor: Leveraging Contextual Changes for Multi-round Code Auto-editing. arXiv:2305.18584 [cs.SE]

[86] Jiayi Wei, Greg Durrett, and Isil Dillig. 2023. TypeT5: Seq2seq Type Inference using Static Analysis. arXiv:2303.09564 [cs.SE]

[87] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems* 35 (2022), 24824–24837.

[88] Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. 2023. Automated program repair in the era of large pre-trained language models. In *Proceedings of the 45th International Conference on Software Engineering (ICSE 2023). Association for Computing Machinery.*

[89] Frank F. Xu, Uri Alon, Graham Neubig, and Vincent Josua Hellendoorn. 2022. A Systematic Evaluation of Large Language Models of Code. In *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming (MAPS 2022).* Association for Computing Machinery, New York, NY, USA, 1–10. https://doi.org/10.1145/3520312.3534862 event-place: San Diego, CA, USA.

[90] Frank F Xu, Junxian He, Graham Neubig, and Vincent J Hellendoorn. 2021. Capturing structural locality in non-parametric language models. *arXiv preprint arXiv:2110.02870* (2021).

[91] Shengzhe Xu, Ziqi Dong, and Na Meng. 2019. Meditor: inference and application of API migration edits. In *2019 IEEE/ACM 27th International Conference on Program Comprehension (ICPC).* IEEE, 335–346.

[92] Pengcheng Yin, Graham Neubig, Miltiadis Allamanis, Marc Brockschmidt, and Alexander Gaunt. 2019. Learning to Represent Edits. In *ICLR 2019.* https://www.microsoft.com/en-us/research/publication/learning-to-represent-edits/ arXiv:1810.13337 [cs.LG].

[93] Jyh-shiarn Yur, Barbara G Ryder, and William A Landi. 1999. An incremental flow-and context-sensitive pointer aliasing analysis. In *Proceedings of the 21st International conference on Software Engineering.* 442–451.

[94] Fengji Zhang, Bei Chen, Yue Zhang, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. 2023. RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation. *arXiv preprint arXiv:2303.12570* (2023).

[95] Shun Zhang, Zhenfang Chen, Yikang Shen, Mingyu Ding, Joshua B. Tenenbaum, and Chuang Gan. 2023. Planning with Large Language Models for Code Generation. arXiv:2303.05510 [cs.LG]

[96] Yuhao Zhang, Yasharth Bajpai, Priyanshu Gupta, Ameya Ketkar, Miltiadis Allamanis, Titus Barik, Sumit Gulwani, Arjun Radhakrishna, Mohammad Raza, Gustavo Soares, and Ashish Tiwari. 2022. Overwatch: Learning patterns in code edit sequences. *Proc. ACM Program. Lang.* 6, OOPSLA2 (2022), 395–423. https://doi.org/10.1145/3563302

Received 2023-09-29; accepted 2024-01-23